

Project Proposal

Title

Deep Q-Learning and Planning in Self-Play Mini-Chess: A Comparison of DQN, Double DQN, and MCTS

Team(1 person)

Zhenkang Dong

1. Problem Description

This project trains a self-play agent in 5×5 Gardner Mini-Chess (10 pieces per side: K/Q/R/B/N/5P) and conducts a controlled comparison between DQN, Double DQN, and MCTS in terms of convergence behavior and stability. The [Mini-Chess](#) environment is based on an open-source 5×5 Gardner Mini-Chess implementation used solely for rule enforcement and move generation. Given limited hardware (RTX 4060 Laptop GPU) and a solo project scope, 5×5 Mini-Chess is selected as a computationally manageable yet strategically rich domain.

States are encoded as 12-channel one-hot planes (own/opponent $\times \{K, Q, R, B, N, P\}$) of size 5×5 . The action space consists of 625 actions (from-square $\times$ to-square) with legal-action masking. Rewards are terminal only (+1 win, -1 loss, 0 draw). A move limit (e.g., 80/100) prevents infinite loops.

Gardner 5×5 has been weakly solved (game-theoretic value: draw), although first-move advantage exists. To mitigate bias, starting sides alternate during training and evaluation, and win rates are reported separately for each side. The objective is to compare learning behavior rather than solve the game optimally.

2. Algorithms

Three methods will be implemented in PyTorch with GPU acceleration: DQN, Double DQN, and MCTS.

DQN serves as the baseline. Double DQN is compared under identical architecture, hyperparameters, and training budget. Evaluation is conducted against fixed random and heuristic opponents.

Mini-Chess has a discrete action space (625 actions) and sparse terminal rewards. DQN is a standard deep value-learning method for discrete control, and Double DQN addresses overestimation bias, which may affect stability in self-play.

DQN is typically applied to single-agent MDPs (e.g., Atari). Here, it is applied to a two-player deterministic game via frozen-opponent self-play and legal-action masking.

MCTS serves as a planning-based method that selects actions via simulated rollouts. A plain MCTS implementation is used with a simple random rollout policy (random legal moves until termination) to estimate outcomes.

Deep RL has been widely applied to board games; while strong chess systems rely on search-based methods (e.g., AlphaZero), value-based approaches provide a controlled framework for analyzing learning stability in adversarial settings. The comparison isolates the effect of target estimation under identical architectures and training budgets.

3. Results

Double DQN is expected to produce more stable learning curves and lower variance across random seeds. DQN may exhibit stronger overestimation and oscillatory behavior. In contrast, MCTS is expected to provide more consistent decision quality without training, although it may require higher computation per move.

Metrics include win-rate curves over training, average return, variance across seeds, and evaluation against random and heuristic opponents (prioritize checks; else prioritize captures; else random legal move.).

Risks include sparse terminal rewards and instability from self-play. If convergence is insufficient, the project will revert to a reduced piece configuration (K+R+3P) or a 4×4 board to ensure demonstrable algorithmic differences.